

Institut Supérieur du Génie Appliqué (IGA)

Rapport de Fin de Module

Application Mobile

Gestion de Cabinet Dentaire

Android • Java • SQLite • Agile Scrum

Réalisée par

HADDAD Hajar
ELFARKCHI Meriame

Encadré par

M. Baheri

Année Universitaire 2026–2027

Table des matières

1	Cadrage du Projet	3
1.1	Introduction	3
1.2	Contexte	3
1.2.1	Digitalisation du Secteur Médical	3
1.2.2	Adoption des Smartphones dans les Cabinets Médicaux	3
1.2.3	Limites des Solutions Actuelles	3
1.2.4	Opportunités Technologiques	4
1.3	Problématique	4
1.4	Objectifs	4
1.5	Justification	5
2	Méthodologie : Agile Scrum	6
2.1	Présentation de la Méthode Agile	6
2.1.1	Qu'est-ce que l'Agilité ?	6
2.1.2	Pourquoi Avons-nous Choisi Agile Scrum ?	6
2.2	Le Framework Scrum	7
2.2.1	Les Rôles dans Scrum	7
2.2.2	Les Artefacts Scrum	7
2.2.3	Les Cérémonies Scrum	7
2.2.4	Le Sprint : Unité de Travail Fondamentale	8
2.3	Product Backlog et Planification des Sprints	8
2.3.1	Product Backlog Initial	8
2.3.2	Planification des Sprints	9
2.4	Outils Techniques Utilisés	9
2.4.1	Android Studio	9
2.4.2	Java pour Android	9
2.4.3	SQLite	10
3	Analyse des Besoins	11
3.1	Introduction	11
3.2	Identification des Acteurs	11
3.3	Besoins Fonctionnels par Espace	11
3.3.1	Espace Patient	11
3.3.2	Espace Secrétaire	12
3.3.3	Espace Dentiste	12
3.4	Besoins Non Fonctionnels	13
4	Conception et Modélisation	14
4.1	Introduction	14
4.2	Architecture de l'Application	14
4.3	Diagrammes de Cas d'Utilisation	14
4.3.1	Diagramme de Cas d'Utilisation — Patient	14
4.3.2	Diagramme de Cas d'Utilisation — Secrétaire	15
4.3.3	Diagramme de Cas d'Utilisation — Dentiste	16
4.4	Diagramme de Classes	18
4.4.1	La Classe Abstraite User (Classe Mère)	18

4.4.2	Héritage : Dentiste, Secrétaire, Patient	19
4.4.3	La Classe Service	19
4.4.4	La Classe Planning	19
4.4.5	La Classe RendezVous	19
4.4.6	La Classe Paiement	20
4.4.7	La Classe Avis	20
4.4.8	Les Énumérations	20
5	Réalisation	21
5.1	Structure du Projet Android	21
5.2	Implémentation de la Base de Données SQLite	21
5.3	Exemple de DAO — PatientDAO	24
5.4	Exemple d'Activity — LoginActivity	26
6	Conclusion	29
6.1	Bilan du Projet	29
6.2	Compétences Acquisées	29
6.3	Perspectives d'Amélioration	29

1 Cadrage du Projet

1.1 Introduction

Ce chapitre pose le cadre général de notre projet. Il présente le contexte socio-technologique dans lequel s'inscrit le développement de notre application mobile, identifie la problématique centrale à laquelle nous répondons, et formule les objectifs stratégiques que nous nous sommes fixés. Il se conclut par une justification de la pertinence et de la valeur ajoutée du projet.

1.2 Contexte

1.2.1 Digitalisation du Secteur Médical

La transformation numérique des établissements de santé est aujourd'hui une priorité mondiale. Les cabinets dentaires, longtemps dépendants de registres papier ou de logiciels de bureau peu flexibles, font face à un besoin croissant d'outils modernes, accessibles, mobiles et autonomes (c'est-à-dire fonctionnant sans connexion Internet permanente).

Dans ce contexte, les applications mobiles Android représentent une solution idéale : elles sont portables, intuitives, économiques à développer et déployables immédiatement sur les smartphones des professionnels de santé.

1.2.2 Adoption des Smartphones dans les Cabinets Médicaux

Au Maroc, l'adoption des smartphones dans le milieu médical s'accélère, portée par :

- La généralisation des smartphones Android accessibles à toutes les catégories socio-professionnelles.
- L'amélioration de la couverture réseau mobile sur l'ensemble du territoire.
- Une sensibilisation croissante des professionnels de santé aux avantages du numérique.
- Une demande forte des patients pour des services plus réactifs, transparents et accessibles.

1.2.3 Limites des Solutions Actuelles

Malgré ce contexte favorable, les cabinets dentaires marocains souffrent encore de nombreuses lacunes organisationnelles :

- **Gestion manuelle des rendez-vous** : carnets papier, risques d'erreurs, chevauchements de créneaux et pertes d'information fréquentes.
- **Absence de dossiers patients numériques** : l'historique médical est fragmenté, difficile à consulter rapidement lors d'une consultation.
- **Communication inefficace** : les patients ne reçoivent pas systématiquement de rappels, ce qui génère un taux élevé d'absences non justifiées.
- **Suivi financier approximatif** : la traçabilité des paiements repose sur des cahiers manuscrits, source d'erreurs et d'impayés non détectés.
- **Manque de coordination** : le dentiste, la secrétaire et le patient utilisent des canaux de communication séparés et non synchronisés.

1.2.4 Opportunités Technologiques

Plusieurs facteurs technologiques rendent ce projet particulièrement opportun :

- La plateforme **Android** est mature, largement documentée et accessible pour le développement Java.
- **SQLite** offre une base de données relationnelle légère, embarquée directement dans l'application, sans besoin de serveur.
- **Android Studio** fournit un environnement de développement complet, avec émulateur, débogueur et éditeur visuel de layouts.
- La **méthodologie Agile Scrum** permet de livrer des fonctionnalités opérationnelles de manière incrémentale, réduisant les risques projet.

1.3 Problématique

Question centrale

Comment concevoir et développer une application mobile Android permettant à un cabinet dentaire de gérer efficacement ses patients, ses rendez-vous, son planning, ses services et ses paiements, tout en offrant une expérience fluide, sécurisée et cohérente aux trois acteurs principaux : le Dentiste, la Secrétaire et le Patient ?

Cette problématique se décline autour de cinq axes :

1. **Centralisation des données patients** : Comment stocker, consulter et mettre à jour efficacement les informations médicales et les historiques des patients dans une base de données locale ?
2. **Gestion intelligente des rendez-vous** : Comment prévenir les conflits de planning, faciliter la prise de rendez-vous en ligne et assurer un suivi du statut de chaque consultation ?
3. **Coordination entre acteurs** : Comment permettre au Dentiste, à la Secrétaire et au Patient d'interagir de manière cohérente via une même application, chacun disposant d'un espace adapté à son rôle ?
4. **Suivi financier automatisé** : Comment enregistrer, tracer et catégoriser les paiements associés aux consultations, en supportant plusieurs méthodes de règlement ?
5. **Accessibilité hors ligne** : Comment garantir le fonctionnement de l'application sans connexion Internet, en s'appuyant uniquement sur le stockage local SQLite ?

1.4 Objectifs

Notre projet poursuit cinq objectifs complémentaires :

1. **Digitaliser la gestion des patients** : offrir une interface complète pour créer, modifier et consulter les dossiers patients, incluant l'historique médical.
2. **Optimiser la planification** : permettre une gestion fluide des créneaux horaires, des disponibilités du dentiste et des rendez-vous associés, avec détection automatique des conflits.

3. **Sécuriser les accès par rôle** : implémenter un système d'authentification différencié (Patient, Secrétaire, Dentiste) garantissant que chaque acteur n'accède qu'aux fonctionnalités qui le concernent.
4. **Automatiser la gestion financière** : enregistrer les paiements, suivre leur statut (EN_ATTENTE, PAYÉ, ANNULÉ) et prendre en charge plusieurs méthodes de règlement (espèces, carte, en ligne).
5. **Améliorer l'expérience patient** : permettre la prise de rendez-vous autonome, l'envoi de notifications de rappel et la possibilité de noter les services reçus.

1.5 Justification

Ce projet se justifie par plusieurs raisons convergentes :

- **Besoin réel et non satisfait** : aucune solution mobile légère, autonome et adaptée aux cabinets dentaires marocains n'existe aujourd'hui sur le marché local.
- **Valeur pédagogique forte** : ce projet mobilise l'ensemble des compétences acquises en développement mobile (Java, Android Studio, SQLite), en modélisation logicielle (UML) et en gestion de projet (Agile Scrum).
- **Fonctionnement hors ligne** : grâce à SQLite, l'application est totalement autonome et ne dépend d'aucune infrastructure serveur externe.
- **Impact professionnel direct** : l'application améliore la productivité du cabinet, réduit les erreurs administratives et augmente la satisfaction des patients.
- **Scalabilité** : l'architecture choisie permet une évolution future vers une synchronisation cloud ou une version multi-cabinets.

2 Méthodologie : Agile Scrum

2.1 Présentation de la Méthode Agile

2.1.1 Qu'est-ce que l'Agilité ?

L'**Agilité** est une approche philosophique et pratique du développement logiciel qui repose sur l'adaptation continue aux changements, la collaboration étroite avec les parties prenantes et la livraison fréquente de logiciels fonctionnels. Elle s'oppose aux méthodes classiques dites "en cascade" (*Waterfall*), dans lesquelles toutes les phases du projet (analyse, conception, développement, test) sont réalisées séquentiellement et rigidement.

L'agilité est formalisée dans le **Manifeste Agile** (2001), qui pose quatre valeurs fondamentales :

Les 4 Valeurs du Manifeste Agile

1. **Les individus et leurs interactions** *plutôt que* les processus et les outils.
2. **Des logiciels opérationnels** *plutôt qu'une* documentation exhaustive.
3. **La collaboration avec les clients** *plutôt que* la négociation contractuelle.
4. **L'adaptation au changement** *plutôt que* le suivi rigide d'un plan initial.

Ces valeurs sont complétées par **12 principes** parmi lesquels :

- Livrer fréquemment des logiciels fonctionnels (toutes les 1 à 4 semaines).
- Accueillir positivement les changements de besoins, même tardivement.
- Maintenir un rythme de développement soutenable.
- Favoriser l'auto-organisation des équipes.
- Réfléchir régulièrement aux améliorations possibles du processus.

2.1.2 Pourquoi Avons-nous Choisi Agile Scrum ?

Pour notre projet d'application mobile de gestion de cabinet dentaire, la méthode Agile Scrum s'est imposée pour plusieurs raisons :

- **Évolutivité des besoins** : les besoins d'un cabinet dentaire peuvent évoluer au fil du développement (ajout d'une fonctionnalité de paiement en ligne, intégration de notifications, etc.). Scrum permet d'intégrer ces changements sans remettre en cause l'ensemble du projet.
- **Visibilité permanente** : les revues de sprint permettent de disposer à tout moment d'une version opérationnelle de l'application, même partielle.
- **Gestion des priorités** : le *Product Backlog* permet de classer les fonctionnalités par ordre de valeur et de se concentrer d'abord sur les fonctionnalités les plus importantes.
- **Détection rapide des erreurs** : les tests sont réalisés à chaque sprint, ce qui évite l'accumulation de bugs.
- **Équipe de petite taille** : Scrum est particulièrement adapté aux équipes de 2 à 9 personnes, ce qui correspond exactement à notre contexte (Hajar et Meriame).

2.2 Le Framework Scrum

2.2.1 Les Rôles dans Scrum

Scrum définit trois rôles principaux :

TABLE 1 – Rôles Scrum dans notre projet

Rôle Scrum	Correspondance	Responsabilités
Product Owner	Représentant du cabinet dentaire	Définit les besoins, priorise le Product Backlog, valide les livrables
Scrum Master	Hajar / Meriame (tournant)	Facilite les cérémonies Scrum, lève les obstacles, garantit l'application du framework
Development Team	Hajar + Meriame	Conçoit, développe et teste les fonctionnalités à chaque sprint

2.2.2 Les Artefacts Scrum

Scrum s'appuie sur trois artefacts principaux :

- **Product Backlog** : liste ordonnée et exhaustive de toutes les fonctionnalités souhaitées pour l'application. Chaque élément est exprimé sous forme de *User Story* (ex : “En tant que patient, je veux pouvoir réserver un rendez-vous afin de choisir le créneau qui me convient.”).
- **Sprint Backlog** : sous-ensemble du Product Backlog sélectionné pour être réalisé lors d'un sprint donné. Il représente l'engagement de l'équipe pour la durée du sprint.
- **Increment** : version fonctionnelle et potentiellement livrable de l'application, produite à la fin de chaque sprint. Chaque incrément s'appuie sur les précédents.

2.2.3 Les Cérémonies Scrum

Scrum organise le travail autour de quatre cérémonies régulières :

TABLE 2 – Cérémonies Scrum

Cérémonie	Durée	Objectif
Sprint Planning	1–2h	Sélectionner les User Stories du Product Backlog à réaliser, définir les tâches techniques et estimer l'effort
Daily Stand-up	15 min	Point quotidien : qu'est-ce que j'ai fait hier ? Que vais-je faire aujourd'hui ? Quels obstacles je rencontre ?
Sprint Review	1h	Démonstration des fonctionnalités développées au Product Owner, recueil des retours et validation
Sprint Retrospective	30–45 min	Analyse du sprint écoulé : ce qui a bien fonctionné, ce qui peut être amélioré dans le prochain sprint

2.2.4 Le Sprint : Unité de Travail Fondamentale

Un **Sprint** est une itération de durée fixe (généralement 1 à 4 semaines) à l'issue de laquelle un incrément fonctionnel est produit. Dans notre projet, nous avons opté pour des sprints de **2 semaines**, équilibre idéal entre rapidité de livraison et profondeur des fonctionnalités.

2.3 Product Backlog et Planification des Sprints

2.3.1 Product Backlog Initial

TABLE 3 – Product Backlog — User Stories principales

ID	User Story	Priorité	Sprint
US01	En tant qu'utilisateur, je veux m'inscrire et me connecter avec mon rôle	Haute	1
US02	En tant que secrétaire, je veux ajouter un nouveau patient	Haute	2
US03	En tant que secrétaire, je veux modifier les informations d'un patient	Haute	2
US04	En tant que secrétaire, je veux créer un rendez-vous pour un patient	Haute	3
US05	En tant que dentiste, je veux définir mes créneaux de disponibilité	Haute	3
US06	En tant que patient, je veux réserver un rendez-vous en ligne	Moyenne	3
US07	En tant que dentiste, je veux ajouter et gérer mes services	Haute	4
US08	En tant que secrétaire, je veux enregistrer un paiement	Moyenne	4
US09	En tant que patient, je veux recevoir des notifications	Moyenne	4
US10	En tant que patient, je veux noter un service reçu	Basse	4

2.3.2 Planification des Sprints

TABLE 4 – Planification détaillée des Sprints

Sprint	Durée	Fonctionnalités livrées
Sprint 1	2 semaines	Authentification (inscription, connexion, déconnexion), gestion des rôles (PATIENT, SECRÉTAIRE, DENTISTE), navigation principale et structure de l'application, mise en place de la base de données SQLite
Sprint 2	2 semaines	Gestion des patients : ajout, modification, consultation de la liste, affichage de l'historique médical
Sprint 3	2 semaines	Gestion du planning (définition des créneaux, blocage, modification), création et annulation des rendez-vous, consultation des disponibilités
Sprint 4	2 semaines	Gestion des services du cabinet (CRUD), enregistrement des paiements, système de notifications, notation des services par les patients
Sprint 5	1 semaine	Tests unitaires et fonctionnels, corrections de bugs, revue finale du code, rédaction du rapport

2.4 Outils Techniques Utilisés

2.4.1 Android Studio

Android Studio est l'environnement de développement intégré (IDE) officiel pour Android, développé par Google et basé sur IntelliJ IDEA. Il offre :

- Un éditeur de code Java/Kotlin intelligent avec complétion automatique et refactoring.
- Un **Layout Editor** visuel pour concevoir les interfaces XML (drag-and-drop).
- Un **émulateur Android** intégré pour tester l'application sans appareil physique.
- Des outils de **débogage** avancés (breakpoints, inspection de la mémoire, Logcat).
- L'intégration de **Gradle** pour la gestion des dépendances et la compilation.
- Un **gestionnaire AVD** (Android Virtual Device) pour simuler différentes tailles d'écrans.

2.4.2 Java pour Android

Java est le langage historique d'Android et a été notre choix pour ce projet. Il présente les avantages suivants dans le contexte mobile :

- Langage **orienté objet** mature, permettant une modélisation précise des entités du domaine (Patient, Dentiste, RendezVous, etc.).

- **Large communauté** et abondance de ressources, tutoriels et bibliothèques Android.
- Excellente **intégration avec le SDK Android** (Activities, Fragments, Services, Intents).
- Gestion automatique de la mémoire via le **Garbage Collector**.
- Support de l'ensemble des **API Android** depuis la version 1.0.
- Compatible avec toutes les versions d'Android ciblées (API 21+).

2.4.3 SQLite

SQLite est le système de gestion de base de données relationnelle (SGBDR) embarqué natif d'Android. C'est une solution de persistance locale qui présente des caractéristiques essentielles pour notre application :

- **Embarqué et autonome** : la base de données est un fichier unique stocké sur l'appareil, aucun serveur n'est nécessaire.
- **Léger et performant** : conçu pour les environnements à ressources limitées (mémoire, processeur, batterie).
- **SQL complet** : supporte les opérations CREATE, INSERT, UPDATE, DELETE, SELECT, les transactions, les clés étrangères et les index.
- **Fonctionnement hors ligne** : toutes les données sont disponibles sans connexion Internet.
- **Intégration Android native** : Android fournit la classe `SQLiteOpenHelper` pour gérer la création, la version et la migration de la base de données.
- **Sécurité** : la base de données est stockée dans le répertoire privé de l'application, inaccessible aux autres applications sans root.

TABLE 5 – Récapitulatif de l'environnement technique

Outil / Technologie	Version	Rôle dans le projet
Android Studio	Hedgehog+	IDE de développement, débogage, émulation
Java (JDK)	17	Langage de programmation principal
SQLite	3.x (natif)	Base de données locale embarquée
Gradle	8.x	Gestionnaire de build et dépendances
Git	2.x	Contrôle de version du code source
Material Design	1.x	Bibliothèque UI composants Android

3 Analyse des Besoins

3.1 Introduction

L'analyse des besoins est une étape fondamentale dans tout projet logiciel. Elle consiste à identifier précisément ce que le système doit faire (besoins fonctionnels) et les contraintes de qualité qu'il doit respecter (besoins non fonctionnels). Cette analyse a été menée en se basant sur des entretiens simulés avec les futurs utilisateurs (dentiste, secrétaire, patient) et sur l'observation des pratiques actuelles dans les cabinets dentaires.

3.2 Identification des Acteurs

Notre application implique trois acteurs principaux, chacun disposant de droits et de fonctionnalités spécifiques :

TABLE 6 – Acteurs du système

Acteur	Description et responsabilités
Patient	Utilisateur final du service. Il s'inscrit, prend ses rendez-vous de manière autonome, consulte son historique, reçoit des notifications et note les services reçus.
Secrétaire	Personnel administratif du cabinet. Elle gère les dossiers patients, le planning, les rendez-vous et les paiements au quotidien.
Dentiste	Professionnel de santé. Il consulte l'historique médical de ses patients, gère son planning de disponibilité, confirme ou annule les rendez-vous et définit les services proposés par le cabinet.

3.3 Besoins Fonctionnels par Espace

3.3.1 Espace Patient

Authentification

- S'inscrire sur la plateforme avec ses informations personnelles.
- Se connecter avec ses identifiants (email / mot de passe).
- Se déconnecter de l'application.

Recherche et Réservation

- Rechercher un service proposé par le cabinet (par nom ou description).
- Consulter les disponibilités du dentiste en temps réel.
- Réserver un rendez-vous en choisissant un service et un créneau disponible.
- Payer en ligne lors de la réservation (optionnel).

Gestion des Rendez-vous (RDV)

- Consulter la liste de tous ses rendez-vous (passés et à venir).
- Modifier un rendez-vous (changement de créneau).
- Annuler un rendez-vous.

Interaction

- Recevoir des notifications de confirmation, de rappel et d'annulation.
- Laisser un avis (note + commentaire) sur un service reçu.

3.3.2 Espace Secrétaire**Authentification**

- Se connecter avec le rôle secrétaire.
- Se déconnecter de l'application.

Gestion des Patients

- Ajouter un nouveau patient (informations personnelles + médicales).
- Modifier les informations d'un patient existant.
- Consulter et rechercher dans la liste des patients.

Gestion du Planning

- Consulter le planning général du cabinet.
- Modifier un créneau horaire.
- Gérer les conflits de planification.

Gestion des Rendez-vous

- Créer un rendez-vous pour un patient (manuellement).
- Modifier un rendez-vous existant.
- Annuler un rendez-vous.
- Consulter la liste complète des rendez-vous.

Gestion des Paiements

- Enregistrer un paiement associé à un rendez-vous terminé.
- Choisir la méthode de paiement (espèces, carte, en ligne).
- Consulter le statut des paiements (en attente, payé, annulé).

3.3.3 Espace Dentiste**Authentification**

- Se connecter avec le rôle dentiste.
- Se déconnecter de l'application.

Suivi Patient

- Consulter l'historique médical complet d'un patient.
- Accéder aux informations personnelles et aux antécédents médicaux.

Gestion des Rendez-vous

- Consulter les rendez-vous du jour et à venir.
- Confirmer un rendez-vous (passage de EN_ATTENTE à CONFIRMÉ).
- Annuler un rendez-vous (avec motif).

Gestion du Planning

- Définir son planning de disponibilité (date, heure début, heure fin).
- Modifier un créneau existant.
- Bloquer des créneaux (ex : congé, formation, urgence).

Gestion des Services

- Ajouter un service proposé (nom, description, prix, durée en minutes).
- Modifier un service existant.
- Supprimer un service.
- Consulter la liste complète des services du cabinet.

3.4 Besoins Non Fonctionnels

TABLE 7 – Besoins non fonctionnels

Critère	Exigence
Performance	Temps de réponse inférieur à 2 secondes pour les opérations courantes (affichage de liste, création d'un RDV). Optimisation des requêtes SQLite via des index sur les colonnes fréquemment interrogées.
Sécurité	Chiffrement des mots de passe (BCrypt ou SHA-256). Gestion stricte des rôles : chaque acteur n'accède qu'à ses fonctionnalités. Protection des données médicales sensibles conformément aux bonnes pratiques.
Disponibilité	Fonctionnement 100% hors ligne grâce à SQLite. Aucune dépendance à Internet pour les opérations de base.
Compatibilité	Compatible Android API 21 (Android 5.0 Lollipop) et toutes les versions supérieures. Testé sur plusieurs tailles d'écrans (5", 6", tablette).
Maintenabilité	Code Java organisé en packages distincts (model, dao, activity, adapter). Respect des conventions de nommage Android. Documentation Javadoc des classes principales.
Ergonomie	Interface conforme aux guidelines <i>Material Design</i> de Google. Navigation intuitive, retour arrière cohérent, messages d'erreur explicites.
Portabilité	Application autonome, déployable via APK sans infrastructure serveur.

4 Conception et Modélisation

4.1 Introduction

Cette section présente la conception architecturale et logicielle de notre application. Nous y exposons les diagrammes de cas d'utilisation pour chacun des trois acteurs, puis le diagramme de classes de l'application. Chaque diagramme est suivi d'une analyse détaillée des éléments représentés et des choix de conception qui en découlent.

4.2 Architecture de l'Application

Notre application mobile repose sur une architecture en trois couches, adaptée aux contraintes d'Android :

- **Couche Présentation (UI)** : Activities et Fragments Android. C'est la couche visible par l'utilisateur. Elle gère l'affichage des données et la capture des interactions (clics, saisies). Exemples : `LoginActivity`, `PatientActivity`, `RendezVousActivity`.
- **Couche Métier (Business Logic)** : Classes Java modélisant les entités du domaine (`User`, `Patient`, `RendezVous`, etc.) et contenant les règles de gestion (ex : vérification des conflits de créneaux, calcul du montant d'un paiement).
- **Couche Données (DAO + SQLite)** : Classes DAO (*Data Access Object*) qui encapsulent toutes les opérations sur la base de données. Elles isolent la logique SQL du reste de l'application, facilitant la maintenance et les tests.

4.3 Diagrammes de Cas d'Utilisation

Les diagrammes de cas d'utilisation (*Use Case Diagrams*) sont des diagrammes comportementaux UML qui représentent les interactions entre les acteurs externes et le système. Ils répondent à la question : **“Qui peut faire quoi sur le système ?”**

Chaque cas d'utilisation (ellipse) représente une fonctionnalité du système du point de vue de l'utilisateur. Les relations **«include»** indiquent qu'un cas d'utilisation en inclut nécessairement un autre pour fonctionner. Les relations **«extend»** représentent des comportements optionnels.

4.3.1 Diagramme de Cas d'Utilisation — Patient

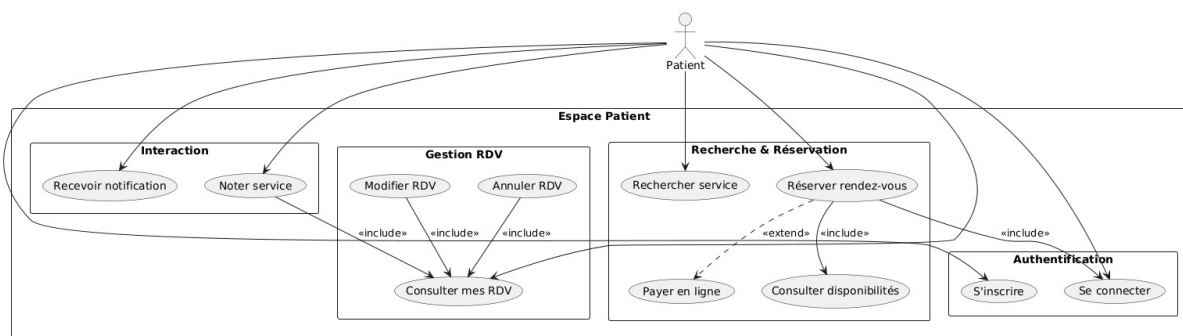


FIGURE 1 – Diagramme de cas d'utilisation — Espace Patient

Analyse détaillée du diagramme Patient :

Ce diagramme représente l'espace fonctionnel complet du patient. Il est organisé en quatre sous-systèmes distincts :

1. **Module Authentification** : Ce module est le point d'entrée obligatoire pour tout patient souhaitant accéder à l'application. Il comprend deux cas d'utilisation :
 - **S'inscrire** : le patient renseigne ses informations personnelles (nom, prénom, email, mot de passe, téléphone, date de naissance, historique médical) pour créer son compte.
 - **Se connecter** : authentification par email et mot de passe. À la connexion réussie, le système identifie le rôle PATIENT et redirige vers le tableau de bord correspondant.
2. **Module Recherche & Réservation** : Ce module permet au patient d'interagir avec l'offre de soins du cabinet :
 - **Réserver rendez-vous** «include» **Consulter disponibilités** : un patient ne peut réserver un rendez-vous qu'après avoir consulté les créneaux disponibles. Cette inclusion est obligatoire.
 - **Rechercher service** : le patient peut filtrer les services par nom ou description avant de réserver.
 - **Payer en ligne** est en relation «extend» avec **Réserver rendez-vous** : le paiement en ligne est optionnel lors de la réservation.
3. **Module Gestion RDV** : Ce module donne au patient une autonomie sur ses rendez-vous :
 - **Modifier RDV** et **Annuler RDV** sont tous deux en relation «include» avec **Consulter mes RDV** : pour modifier ou annuler, le patient doit d'abord consulter la liste de ses rendez-vous.
4. **Module Interaction** : Ce module couvre les interactions sociales du patient avec le cabinet :
 - **Recevoir notification** : rappels automatiques de rendez-vous, confirmations, alertes d'annulation.
 - **Noter service** : après une consultation terminée, le patient peut attribuer une note (1 à 5 étoiles) et laisser un commentaire.

4.3.2 Diagramme de Cas d'Utilisation — Secrétaire

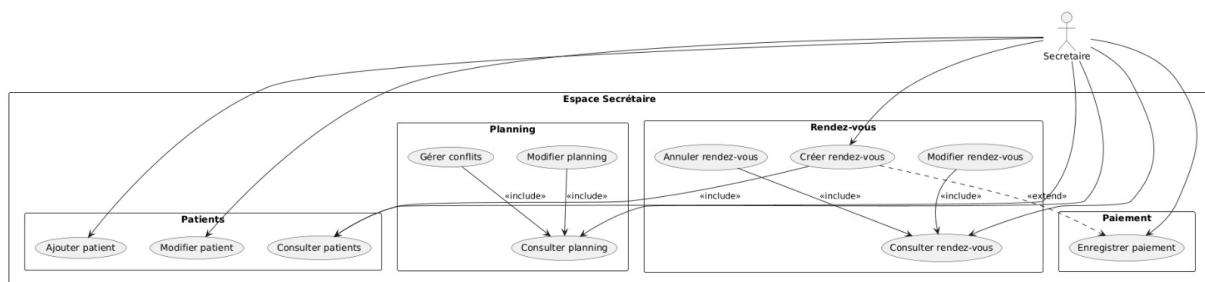


FIGURE 2 – Diagramme de cas d'utilisation — Espace Secrétaire

Analyse détaillée du diagramme Secrétaire :

Ce diagramme représente l'espace administratif de la secrétaire. C'est l'acteur ayant le périmètre fonctionnel le plus large au quotidien. Le diagramme est structuré en quatre modules :

1. **Module Patients** : La secrétaire est responsable de la tenue des dossiers patients :
 - **Ajouter patient** : création d'un nouveau dossier patient dans la base SQLite. Nécessite la saisie d'informations personnelles et médicales.
 - **Modifier patient** : mise à jour des informations d'un patient existant (coordonnées, historique médical).
 - **Consulter patients** : accès à la liste complète des patients avec moteur de recherche.
2. **Module Planning** : La secrétaire co-gère le planning avec le dentiste :
 - **Modifier planning** et **Gérer conflits** sont en relation «include» avec **Consulter planning** : toute action sur le planning nécessite préalablement de le consulter.
 - **Gérer conflits** : détection et résolution des chevauchements de créneaux, par exemple quand deux rendez-vous sont planifiés sur le même créneau.
3. **Module Rendez-vous** : La gestion des rendez-vous est la tâche principale de la secrétaire :
 - **Créer rendez-vous** : création manuelle d'un RDV en choisissant un patient, un service, un créneau disponible et en définissant le motif.
 - **Modifier rendez-vous** : modification d'un RDV existant (changement de créneau, de service ou de motif).
 - **Annuler rendez-vous** : annulation avec enregistrement du motif. Le créneau redevient disponible automatiquement.
 - Les trois cas d'utilisation sont en «include» avec **Consulter rendez-vous**.
 - **Modifier rendez-vous** possède aussi une relation «extend» pointillée vers **Enregistrer paiement**, indiquant que lors d'une modification, l'enregistrement d'un paiement peut optionnellement être déclenché.
4. **Module Paiement** :
 - **Enregistrer paiement** : à la fin d'une consultation, la secrétaire enregistre le montant payé, la méthode (ESPECES, CARTE, EN_LIGNE) et le statut (PAYÉ). Ce cas est en «extend» du module Rendez-vous.

4.3.3 Diagramme de Cas d'Utilisation — Dentiste

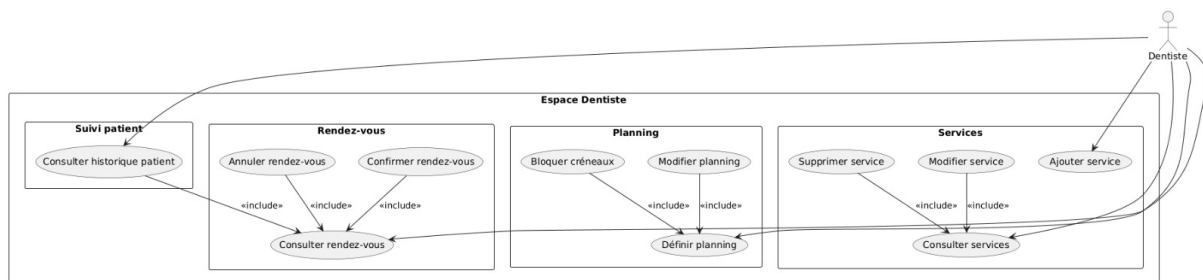


FIGURE 3 – Diagramme de cas d'utilisation — Espace Dentiste

Analyse détaillée du diagramme Dentiste :

Ce diagramme représente l'espace professionnel du dentiste. Il est orienté vers la pratique clinique et la gestion de l'offre de soins. Il comprend quatre modules :

1. **Module Suivi Patient** : Ce module est central pour la pratique clinique du dentiste :
 - **Consulter historique patient** : accès à l'historique médical complet d'un patient (antécédents, consultations précédentes, traitements, notes). Cette fonctionnalité permet au dentiste de préparer chaque consultation.
2. **Module Rendez-vous** : Le dentiste a une vision en lecture/action sur les rendez-vous :
 - **Confirmer rendez-vous** et **Annuler rendez-vous** sont en «include» avec **Consulter rendez-vous** : le dentiste doit d'abord consulter la liste avant d'agir.
 - **Confirmer rendez-vous** : passage du statut EN_ATTENTE à CONFIRMÉ. Déclenche l'envoi d'une notification au patient.
 - **Annuler rendez-vous** : annulation par le dentiste (urgence, indisponibilité). Le créneau est libéré et le patient est notifié.
3. **Module Planning** : Le dentiste est le seul à définir ses disponibilités :
 - **Définir planning** : création de créneaux de disponibilité (date, heure début, heure fin, état DISPONIBLE).
 - **Bloquer créneaux** : marquage de créneaux comme INDISPONIBLE (formation, congé, urgence personnelle).
 - **Modifier planning** : mise à jour d'un créneau existant.
 - Les trois cas sont en «include» avec **Consulter rendez-vous** (par le biais du planning global).
4. **Module Services** : Le dentiste définit l'offre de soins du cabinet :
 - **Ajouter service** : création d'un nouveau service (ex : détartrage, couronne, blanchiment) avec nom, description, prix et durée.
 - **Modifier service** : mise à jour des informations d'un service existant.
 - **Supprimer service** : suppression d'un service (non accessible aux patients une fois supprimé).
 - **Modifier service** et **Supprimer service** sont en «include» avec **Consulter services**.

4.4 Diagramme de Classes

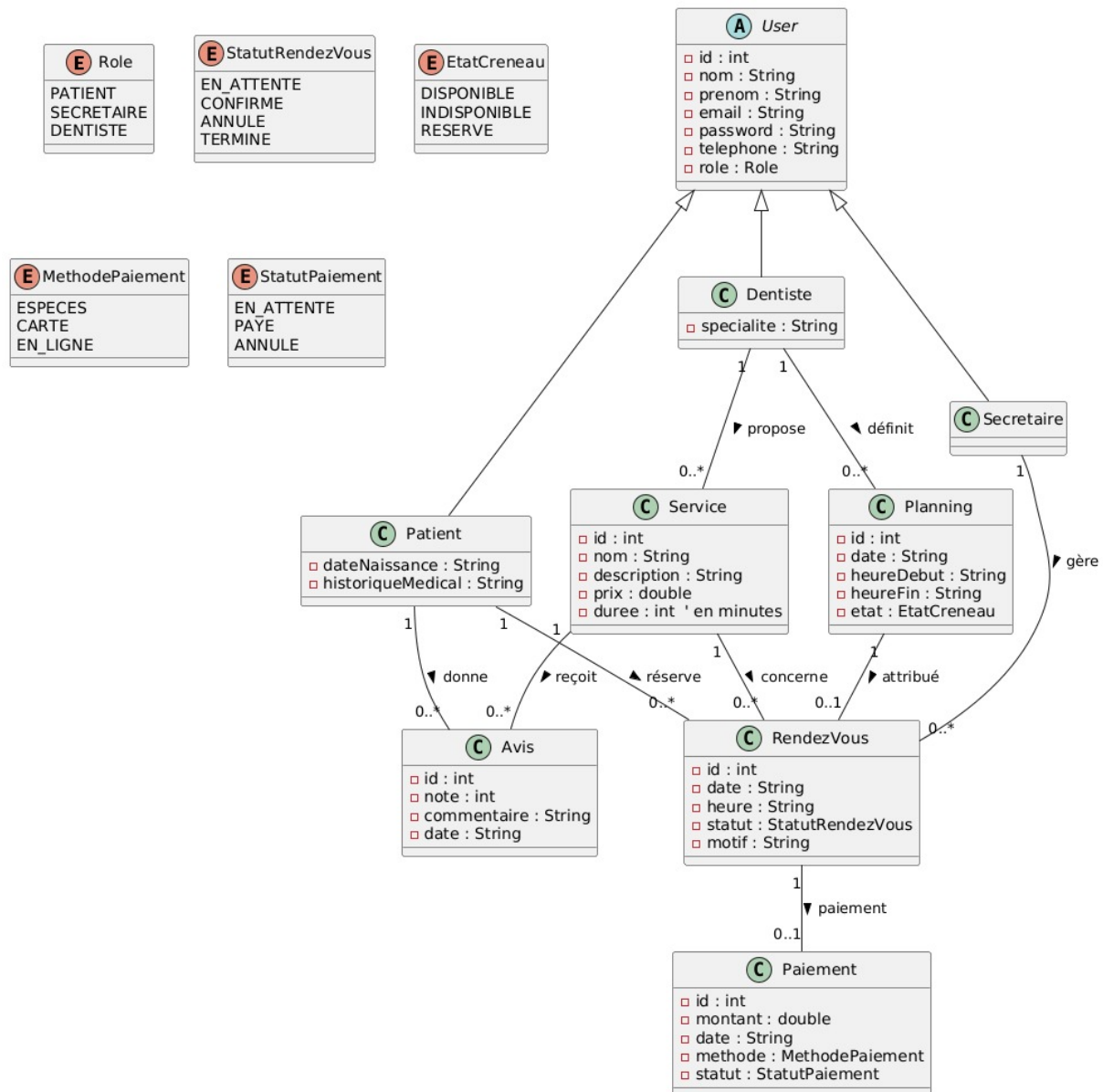


FIGURE 4 – Diagramme de classes de l’application de gestion de cabinet dentaire

Analyse détaillée du diagramme de classes :

Le diagramme de classes est un diagramme structurel UML qui représente la **structure statique** du système : les classes, leurs attributs, leurs méthodes et les relations qui les lient. C’est le schéma de référence pour implémenter les modèles Java et la base de données SQLite.

4.4.1 La Classe Abstraite User (Classe Mère)

User est marquée **Abstract** (A) car elle ne peut pas être instanciée directement. Elle regroupe les attributs communs à tous les utilisateurs :

- `id : int` — identifiant unique auto-incrémenté.

- `nom`, `prenom` : `String` — identité civile.
- `email` : `String` — identifiant unique de connexion.
- `password` : `String` — mot de passe haché.
- `telephone` : `String` — contact.
- `role` : `Role` — énumération déterminant les droits d'accès (PATIENT, SECRÉTAIRE, DENTISTE).

4.4.2 Héritage : Dentiste, Secrétaire, Patient

Les trois classes héritent de `User` (flèches d'héritage). Chacune ajoute des attributs spécifiques :

- `Dentiste` : `specialite` : `String` (ex : orthodontie, chirurgie dentaire).
- `Secrétaire` : aucun attribut supplémentaire dans cette version.
- `Patient` : `dateNaissance` : `String` et `historiqueMedical` : `String` (antécédents, allergies, traitements en cours).

4.4.3 La Classe Service

Représente un soin proposé par le cabinet :

- `id`, `nom`, `description` — identification du service.
- `prix` : `double` — tarif en dirhams.
- `duree` : `int` — durée en minutes (utile pour calculer la fin d'un créneau).

Relation : `Dentiste` **propose** 0..* `services` (un dentiste peut avoir plusieurs services).

4.4.4 La Classe Planning

Représente un créneau horaire dans l'agenda du cabinet :

- `date`, `heureDebut`, `heureFin` : `String` — définissent le créneau temporel.
- `etat` : `EtatCreneau` — DISPONIBLE, INDISPONIBLE ou RESERVÉ.

Relations : `Secrétaire` **gère** 0..* `plannings` (relation 1-1 en réalité, car il n'y a qu'une secrétaire). `Dentiste` **définit** 0..* `plannings`.

4.4.5 La Classe RendezVous

Classe centrale du système, elle représente une consultation planifiée :

- `date`, `heure` : `String` — moment de la consultation.
- `statut` : `StatutRendezVous` — EN_ATTENTE, CONFIRMÉ, ANNULÉ ou TERMINÉ.
- `motif` : `String` — raison de la consultation.

Relations multiples :

- `Patient` **réserve** 0..* `rendez-vous`.
- `RendezVous` **concerne** 0..1 `service` (un RDV peut avoir un service associé).
- `RendezVous` est **attribué** à 0..* `plannings` (le créneau horaire).
- `RendezVous` a 0..1 `paiement` associé.

4.4.6 La Classe Paiement

Enregistre le règlement d'une consultation :

- **montant** : `double` — somme réglée.
- **date** : `String` — date du paiement.
- **methode** : `MethodePaiement` — ESPECES, CARTE ou EN_LIGNE.
- **statut** : `StatutPaiement` — EN_ATTENTE, PAYÉ ou ANNULÉ.

4.4.7 La Classe Avis

Permet aux patients d'évaluer les soins reçus :

- **note** : `int` — notation de 1 à 5.
- **commentaire** : `String` — retour qualitatif du patient.
- **date** : `String` — date de l'avis.

Relation : Patient **donne** 0..* avis. Patient **reçoit** 0..* avis (via les soins reçus).

4.4.8 Les Énumérations

Le diagramme définit cinq énumérations (marquées E) qui contraignent les valeurs possibles de certains attributs :

- **Role** : PATIENT, SECRÉTAIRE, DENTISTE — détermine les droits d'accès.
- **StatutRendezVous** : EN_ATTENTE, CONFIRMÉ, ANNULÉ, TERMINÉ — cycle de vie d'un RDV.
- **EtatCreneau** : DISPONIBLE, INDISPONIBLE, RÉSERVÉ — état d'un créneau du planning.
- **MethodePaiement** : ESPÈCES, CARTE, EN_LIGNE — modes de règlement acceptés.
- **StatutPaiement** : EN_ATTENTE, PAYÉ, ANNULÉ — état du règlement.

5 Réalisation

5.1 Structure du Projet Android

Le projet Android est organisé selon l'architecture standard recommandée par Google :

- app/src/main/java/com/cabinet/dentaire/
 - model/ — Classes Java : User, Patient, Dentiste, Secrétaire, RendezVous, Service, Planning, Paiement, Avis
 - dao/ — Accès SQLite : UserDAO, PatientDAO, RendezVousDAO, ServiceDAO, PlanningDAO, PaiementDAO, AvisDAO
 - activity/ — Interfaces : LoginActivity, RegisterActivity, PatientDashboardActivity, DentisteDashboardActivity, SecrétaireDashboardActivity, RendezVousActivity, ServiceActivity, PlanningActivity, PaiementActivity
 - adapter/ — RecyclerView : PatientAdapter, RDVAdapter, ServiceAdapter, PlanningAdapter
 - database/ — DatabaseHelper.java
- app/src/main/res/layout/ — Fichiers XML des interfaces
- app/src/main/res/drawable/ — Icônes et images
- app/src/main/AndroidManifest.xml — Configuration de l'application

5.2 Implémentation de la Base de Données SQLite

```

1 package com.cabinet.dentaire.database;
2
3 import android.content.Context;
4 import android.database.sqlite.SQLiteDatabase;
5 import android.database.sqlite.SQLiteOpenHelper;
6
7 public class DatabaseHelper extends SQLiteOpenHelper {
8
9     public static final String DATABASE_NAME = "cabinet_dentaire.db";
10    public static final int DATABASE_VERSION = 1;
11
12    // ---- Noms des tables ----
13    public static final String TABLE_USERS = "users";
14    public static final String TABLE_PATIENTS = "patients";
15    public static final String TABLE_DENTISTES = "dentistes";
16    public static final String TABLE_SERVICES = "services";
17    public static final String TABLE_PLANNINGS = "plannings";
18    public static final String TABLE_RENDEZVOUS = "rendez_vous";
19    public static final String TABLE_PAIEMENTS = "paiements";
20    public static final String TABLE_AVIS = "avis";
21
22    // ---- Création des tables ----
23    private static final String CREATE_USERS =
24        "CREATE TABLE " + TABLE_USERS + " (" +
25        "id INTEGER PRIMARY KEY AUTOINCREMENT, " +

```

```

26      "nom TEXT NOT NULL, " +
27      "prenom TEXT NOT NULL, " +
28      "email TEXT NOT NULL UNIQUE, " +
29      "password TEXT NOT NULL, " +
30      "telephone TEXT, " +
31      "role TEXT NOT NULL CHECK(role IN ('PATIENT','SECRETAIRE','
      DENTISTE'))" +
32      ");";
33
34  private static final String CREATE_PATIENTS =
35      "CREATE TABLE " + TABLE_PATIENTS + " (" +
36      "id INTEGER PRIMARY KEY, " +
37      "dateNaissance TEXT, " +
38      "historiqueMedical TEXT, " +
39      "FOREIGN KEY(id) REFERENCES users(id) ON DELETE CASCADE" +
40      ");";
41
42  private static final String CREATE_DENTISTES =
43      "CREATE TABLE " + TABLE_DENTISTES + " (" +
44      "id INTEGER PRIMARY KEY, " +
45      "specialite TEXT, " +
46      "FOREIGN KEY(id) REFERENCES users(id) ON DELETE CASCADE" +
47      ");";
48
49  private static final String CREATE_SERVICES =
50      "CREATE TABLE " + TABLE_SERVICES + " (" +
51      "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
52      "nom TEXT NOT NULL, " +
53      "description TEXT, " +
54      "prix REAL NOT NULL CHECK(prix >= 0), " +
55      "duree INTEGER NOT NULL CHECK(duree > 0)" +
56      ");";
57
58  private static final String CREATE_PLANNINGS =
59      "CREATE TABLE " + TABLE_PLANNINGS + " (" +
60      "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
61      "date TEXT NOT NULL, " +
62      "heureDebut TEXT NOT NULL, " +
63      "heureFin TEXT NOT NULL, " +
64      "etat TEXT NOT NULL DEFAULT 'DISPONIBLE' " +
65      "CHECK(etat IN ('DISPONIBLE','INDISPONIBLE','RESERVE'))" +
66      ");";
67
68  private static final String CREATE_RENDEZVOUS =
69      "CREATE TABLE " + TABLE_RENDEZVOUS + " (" +
70      "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
71      "date TEXT NOT NULL, " +
72      "heure TEXT NOT NULL, " +
73      "statut TEXT NOT NULL DEFAULT 'EN_ATTENTE' " +
74      "CHECK(statut IN ('EN_ATTENTE','CONFIRME','ANNULE','TERMINE
      ')), " +

```

```

75     "motif TEXT, " +
76     "patientId INTEGER, " +
77     "serviceId INTEGER, " +
78     "planningId INTEGER, " +
79     "FOREIGN KEY(patientId) REFERENCES patients(id) ON DELETE
80     SET NULL, " +
81     "FOREIGN KEY(serviceId) REFERENCES services(id) ON DELETE
82     SET NULL, " +
83     "FOREIGN KEY(planningId) REFERENCES plannings(id) ON DELETE
84     SET NULL" +
85     ");";
86
87     private static final String CREATE_PAIEMENTS =
88     "CREATE TABLE " + TABLE_PAIEMENTS + " (" +
89     "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
90     "montant REAL NOT NULL CHECK(montant >= 0), " +
91     "date TEXT NOT NULL, " +
92     "methode TEXT NOT NULL " +
93     "CHECK(methode IN ('ESPECES','CARTE','EN_LIGNE'))), " +
94     "statut TEXT NOT NULL DEFAULT 'EN_ATTENTE' " +
95     "CHECK(statut IN ('EN_ATTENTE','PAYE','ANNULE'))), " +
96     "rendezVousId INTEGER UNIQUE, " +
97     "FOREIGN KEY(rendezVousId) REFERENCES rendez_vous(id) ON
98     DELETE CASCADE" +
99     ");";
100
101     private static final String CREATE_AVIS =
102     "CREATE TABLE " + TABLE_AVIS + " (" +
103     "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
104     "note INTEGER NOT NULL CHECK(note BETWEEN 1 AND 5), " +
105     "commentaire TEXT, " +
106     "date TEXT NOT NULL, " +
107     "patientId INTEGER, " +
108     "FOREIGN KEY(patientId) REFERENCES patients(id) ON DELETE
109     CASCADE" +
110     ");";
111
112     public DatabaseHelper(Context context) {
113         super(context, DATABASE_NAME, null, DATABASE_VERSION);
114     }
115
116     @Override
117     public void onCreate(SQLiteDatabase db) {
118         db.execSQL("PRAGMA foreign_keys = ON;");
119         db.execSQL(CREATE_USERS);
120         db.execSQL(CREATE_PATIENTS);
121         db.execSQL(CREATE_DENTISTES);
122         db.execSQL(CREATE_SERVICES);
123         db.execSQL(CREATE_PLANNINGS);
124         db.execSQL(CREATE_RENDEZVOUS);
125         db.execSQL(CREATE_PAIEMENTS);

```



```

121         db.execSQL(CREATE_AVIS);
122     }
123
124     @Override
125     public void onUpgrade(SQLiteDatabase db, int oldVersion, int
        newVersion) {
126         db.execSQL("DROP TABLE IF EXISTS " + TABLE_AVIS);
127         db.execSQL("DROP TABLE IF EXISTS " + TABLE_PAIEMENTS);
128         db.execSQL("DROP TABLE IF EXISTS " + TABLE_RENDEZVOUS);
129         db.execSQL("DROP TABLE IF EXISTS " + TABLE_PLANNINGS);
130         db.execSQL("DROP TABLE IF EXISTS " + TABLE_SERVICES);
131         db.execSQL("DROP TABLE IF EXISTS " + TABLE_DENTISTES);
132         db.execSQL("DROP TABLE IF EXISTS " + TABLE_PATIENTS);
133         db.execSQL("DROP TABLE IF EXISTS " + TABLE_USERS);
134         onCreate(db);
135     }
136
137     @Override
138     public void onOpen(SQLiteDatabase db) {
139         super.onOpen(db);
140         if (!db.isReadOnly()) {
141             db.execSQL("PRAGMA foreign_keys = ON;");
142         }
143     }
144 }

```

Listing 1 – DatabaseHelper.java — Création complète de la base de données

5.3 Exemple de DAO — PatientDAO

```

1 package com.cabinet.dentaire.dao;
2
3 import android.content.ContentValues;
4 import android.content.Context;
5 import android.database.Cursor;
6 import android.database.sqlite.SQLiteDatabase;
7 import com.cabinet.dentaire.database.DatabaseHelper;
8 import com.cabinet.dentaire.model.Patient;
9 import java.util.ArrayList;
10 import java.util.List;
11
12 public class PatientDAO {
13
14     private final DatabaseHelper dbHelper;
15
16     public PatientDAO(Context context) {
17         dbHelper = new DatabaseHelper(context);
18     }
19
20     /** Ajouter un nouveau patient (user + patient) */
21     public long addPatient(Patient patient) {

```

```

22     SQLiteDatabase db = dbHelper.getWritableDatabase();
23     db.beginTransaction();
24     try {
25         // 1. Insertion dans users
26         ContentValues userValues = new ContentValues();
27         userValues.put("nom", patient.getNom());
28         userValues.put("prenom", patient.getPrenom());
29         userValues.put("email", patient.getEmail());
30         userValues.put("password", patient.getPassword());
31         userValues.put("telephone", patient.getTelephone());
32         userValues.put("role", "PATIENT");
33         long userId = db.insert("users", null, userValues);
34
35         // 2. Insertion dans patients
36         ContentValues patientValues = new ContentValues();
37         patientValues.put("id", userId);
38         patientValues.put("dateNaissance", patient.
39             getDateNaissance());
40         patientValues.put("historiqueMedical", patient.
41             getHistoriqueMedical());
42         db.insert("patients", null, patientValues);
43
44         db.setTransactionSuccessful();
45         return userId;
46     } finally {
47         db.endTransaction();
48     }
49
50     /** R cup rer tous les patients */
51     public List<Patient> getAllPatients() {
52         List<Patient> list = new ArrayList<>();
53         SQLiteDatabase db = dbHelper.getReadableDatabase();
54         String query =
55             "SELECT u.id, u.nom, u.prenom, u.email, u.telephone, "
56             +
57             "p.dateNaissance, p.historiqueMedical " +
58             "FROM users u INNER JOIN patients p ON u.id = p.id " +
59             "ORDER BY u.nom ASC";
60         Cursor cursor = db.rawQuery(query, null);
61         if (cursor.moveToFirst()) {
62             do {
63                 Patient p = new Patient();
64                 p.setId(cursor.getInt(0));
65                 p.setNom(cursor.getString(1));
66                 p.setPrenom(cursor.getString(2));
67                 p.setEmail(cursor.getString(3));
68                 p.setTelephone(cursor.getString(4));
69                 p.setDateNaissance(cursor.getString(5));
70                 p.setHistoriqueMedical(cursor.getString(6));
71                 list.add(p);

```

```

70         } while (cursor.moveToNext());
71     }
72     cursor.close();
73     return list;
74 }
75
76 /** Mettre à jour un patient */
77 public int updatePatient(Patient patient) {
78     SQLiteDatabase db = dbHelper.getWritableDatabase();
79     ContentValues values = new ContentValues();
80     values.put("dateNaissance", patient.getDateNaissance());
81     ;
82     values.put("historiqueMedical", patient.
83         getHistoriqueMedical());
84     return db.update("patients", values, "id = ?",
85         new String[]{String.valueOf(patient.getId())});
86 }
87
88 /** Supprimer un patient */
89 public void deletePatient(int patientId) {
90     SQLiteDatabase db = dbHelper.getWritableDatabase();
91     // La suppression en cascade supprime aussi la ligne dans
92     patients
93     db.delete("users", "id = ?",
94         new String[]{String.valueOf(patientId)});
95 }

```

Listing 2 – PatientDAO.java — Accès aux données patients

5.4 Exemple d'Activity — LoginActivity

```

1 package com.cabinet.dentaire.activity;
2
3 import android.content.Intent;
4 import android.os.Bundle;
5 import android.widget.Button;
6 import android.widget.EditText;
7 import android.widget.Toast;
8 import androidx.appcompat.app.AppCompatActivity;
9 import com.cabinet.dentaire.dao.UserDAO;
10 import com.cabinet.dentaire.model.User;
11
12 public class LoginActivity extends AppCompatActivity {
13
14     private EditText etEmail, etPassword;
15     private Button btnLogin;
16     private UserDAO userDAO;
17
18     @Override
19     protected void onCreate(Bundle savedInstanceState) {

```

```
20     super.onCreate(savedInstanceState);
21     setContentView(R.layout.activity_login);
22
23     userDAO    = new UserDAO(this);
24     etEmail    = findViewById(R.id.etEmail);
25     etPassword = findViewById(R.id.etPassword);
26     btnLogin   = findViewById(R.id.btnLogin);
27
28     btnLogin.setOnClickListener(v -> login());
29 }
30
31 private void login() {
32     String email    = etEmail.getText().toString().trim();
33     String password = etPassword.getText().toString().trim();
34
35     if (email.isEmpty() || password.isEmpty()) {
36         Toast.makeText(this,
37             "Veuillez remplir tous les champs", Toast.
38                 LENGTH_SHORT).show();
39         return;
40     }
41
42     User user = userDAO.authenticate(email, password);
43     if (user == null) {
44         Toast.makeText(this,
45             "Email ou mot de passe incorrect", Toast.
46                 LENGTH_SHORT).show();
47         return;
48     }
49
50     // Redirection selon le r le
51     Intent intent;
52     switch (user.getRole()) {
53         case "PATIENT":
54             intent = new Intent(this, PatientDashboardActivity.
55                 class);
56             break;
57         case "SECRETAIRE":
58             intent = new Intent(this,
59                 SecretaireDashboardActivity.class);
60             break;
61         case "DENTISTE":
62             intent = new Intent(this, DentisteDashboardActivity.
63                 class);
64             break;
65         default:
66             Toast.makeText(this, "Role inconnu", Toast.
67                 LENGTH_SHORT).show();
68             return;
69     }
70     intent.putExtra("userId", user.getId());
```

```
65     startActivity(intent);  
66     finish(); // Empêche le retour à l'écran de connexion  
67 }  
68 }
```

Listing 3 – LoginActivity.java — Authentification et gestion des rôles

6 Conclusion

6.1 Bilan du Projet

Ce projet nous a permis de concevoir, modéliser et développer une application mobile Android complète dédiée à la gestion d'un cabinet dentaire. En combinant la rigueur de la modélisation UML, la souplesse de la méthode Agile Scrum et la puissance des outils Android (Java, SQLite, Android Studio), nous avons livré une solution fonctionnelle, autonome et adaptée aux besoins réels d'un cabinet dentaire.

L'application répond à l'ensemble des besoins identifiés lors de l'analyse : gestion des patients, planification des rendez-vous, définition des services, enregistrement des paiements et interaction entre les trois acteurs (Dentiste, Secrétaire, Patient).

6.2 Compétences Acquises

La réalisation de ce projet nous a permis de consolider et d'approfondir les compétences suivantes :

- **Développement Android** : maîtrise du cycle de vie des Activities, gestion des RecyclerViews, navigation entre écrans avec Intents, Material Design.
- **Bases de données SQLite** : conception de schémas relationnels, implémentation via SQLiteOpenHelper, pattern DAO, transactions et gestion des intégrités référentielles.
- **Méthode Agile Scrum** : rédaction de User Stories, planification de sprints, revues et rétrospectives, gestion d'un Product Backlog.
- **Modélisation UML** : diagrammes de cas d'utilisation avec relations «include» et «extend», diagramme de classes avec héritage et multiplicités.
- **Gestion de projet** : organisation du travail en binôme, versioning avec Git, documentation technique.

6.3 Perspectives d'Amélioration

Plusieurs évolutions sont envisageables pour les versions futures de l'application :

- **Synchronisation cloud** : intégration d'un backend REST (Spring Boot ou Firebase) pour synchroniser les données entre plusieurs appareils et permettre un accès multi-utilisateurs simultané.
- **Notifications push** : rappels automatiques envoyés aux patients (via Firebase Cloud Messaging) 24h avant leurs rendez-vous.
- **Migration vers Kotlin** : adoption progressive du langage officiellement recommandé par Google, offrant une syntaxe plus concise et des coroutines pour les opérations asynchrones.
- **Génération de rapports PDF** : export des statistiques du cabinet (nombre de consultations, chiffre d'affaires, patients les plus fréquents) en format PDF.
- **Module de téléconsultation** : intégration d'une fonctionnalité de consultation à distance via vidéo.
- **Tableau de bord analytique** : graphiques de suivi de l'activité du cabinet (consultations par mois, taux d'annulation, services les plus demandés).